

Plasticity Without Parameters: Two-Timescale Usage-Statistics Consolidation in a Local-First Agent Harness

Danilo Canivel
Independent / KAOS project
d.canivel@gmail.com

July 23, 2026

Abstract

Agent harnesses accumulate skills, memories, and failure records, yet in most frameworks this state is static: it does not reorganize in response to how it is used. We describe the plasticity architecture of KAOS, an open-source, local-first multi-agent harness in which every runtime artifact lives in a single SQLite file. The design borrows one property from biological neuroplasticity—the separation of timescales between fast synaptic updates and slow structural consolidation—while remaining strictly parameter-free: no fine-tuning, no embeddings, no gradient updates. On the fast path, agent activity writes only raw telemetry (micro-benchmarked at ≈ 0 median overhead, a point estimate within noise); at episode boundaries, a batch pass rebuilds a Hebbian co-occurrence graph; retrieval is re-ranked at query time by Wilson-bound success and recency computed over that telemetry. On two small, fully committed retrieval benchmarks this yields +10.0 and +13.3 percentage points of top-1 accuracy over BM25 ($n=10$ and $n=15$ queries; we discuss why we report such small benchmarks rather than headline them). Equally central is the discipline that has governed development since v0.9: no mechanism candidate could ship without surviving a pre-registered, hash-locked, falsifiable probe, and we report the negative results this produced—including a structural argument that retrieval-side semantic consolidation is infeasible under lexical-only retrieval constraints, and the discovery (by a multi-agent self-audit) of bugs in our own verdict instrument. Ten candidates were evaluated; none shipped without surviving a probe. We argue that for agent harnesses, the evaluation discipline is as much a contribution as any mechanism it gated.

1 Introduction

In June 2026, a submission to the ARC-AGI-3 competition entered the public leaderboard’s top 4% with a score of 0.43; six weeks later the same entry, operated autonomously by its harness with occasional operator configuration, had more than doubled its score and stood at rank #39 of roughly 1,700 teams.¹ The interesting question is not the rank; it is *what, mechanically, was allowed to change between those two dates*. The underlying language models were unchanged and frozen. What changed was harness state: which strategies were retained, how stored procedures were ranked, and which failure patterns were recognized and avoided.

This paper describes that machinery. KAOS is an open-source, local-first multi-agent harness whose entire runtime state—per-agent virtual filesystems, an append-only event journal, checkpoints,

¹Public leaderboard, ARC Prize 2026 (ARC-AGI-3), entry `canivel`; snapshots recorded June 8–July 18, 2026 (Appendix A, P14). We report this as an *observation about a deployment*, not a controlled result: the public leaderboard uses roughly half the test data, the field changed over the period, and no ablation isolates the harness’s contribution. We include it because it motivated the question this paper answers.

cross-agent memory, and a skill library—resides in one SQLite file. Since v0.8 it has included a *plasticity layer*: usage statistics that reorganize retrieval continuously as a side effect of normal operation. The design constraint that shapes everything is that KAOS forbids parameter updates of any kind: no fine-tuning, no LoRA, no embeddings, no learned rerankers. Whatever “learning” occurs must be arithmetic over recorded outcomes.

Two questions structure the paper. First, *how much adaptivity can a harness obtain from usage statistics alone*, and at what runtime cost? Second—because the space of plausible-sounding mechanisms is large and the evidence for each is typically a demo—*how does one keep such a system honest*? Our answer to the second question is a falsifiable-evaluation discipline: every mechanism candidate is evaluated against kill gates that are written and cryptographically locked *before* any feature code exists, with a mandatory falsification self-test and a binding verdict. The discipline produced more rejections than acceptances, and we consider several of those rejections to be the most informative results in this paper.

Our contributions:

- **A two-timescale plasticity architecture** for agent harnesses (§4): inline hooks that write only raw telemetry (median overhead indistinguishable from zero in our micro-benchmark), with Hebbian association building and structural consolidation deferred to episode boundaries. We motivate the split with a measured consolidation cost curve that is prohibitive on the hot path (35.5s at 10,000 skills).
- **A parameter-free retrieval re-ranking rule** (§5) combining BM25 with Wilson lower-bound success rates and recency decay, evaluated on two small, fully committed benchmarks (+10.0 pp adversarial, +13.3 pp realistic), with a sensitivity sweep showing a broad plateau in its single weighting hyperparameter.
- **A falsifiable-evaluation discipline for harness mechanisms** (§6): pre-registered, sha256-locked kill gates; a falsification self-test that renders a harness inadmissible if its feature cannot lose; binding ACCEPT/REJECT/VOID verdicts; and an append-only experiment journal. Ten candidates were evaluated under it; none shipped without surviving a probe.
- **Negative results** (§7): (a) a pre-registered probe that *rejected* LLM-synthesized memory consolidation despite the synthesized artifacts being retrieved (the “lexical vise”: under lexical-only retrieval, a stored abstraction must be simultaneously token-faithful and query-bridging, and the two requirements conflict); (b) a VOID verdict on a pre-execution action-validation layer whose lock forbade synthetic workloads; and (c) instrument-integrity bugs in our own verdict machinery, found by a 48-agent self-audit and fixed test-first.

Everything quantitative in this paper traces to a committed artifact in the public repository; Appendix A maps every claim to its file and, where applicable, its pre-registration hash. The paper itself was drafted and revised through KAOS-orchestrated agent panels (grounding verification, methodological review, and adversarial review), with the panel findings recorded alongside the source (§11).

2 Related Work

Memory and skill accumulation in LLM agents. Voyager [Wang et al., 2023] accumulates verified skills in an ever-growing library; MemGPT [Packer et al., 2023] manages hierarchical memory with LLM-controlled paging; Reflexion [Shinn et al., 2023] converts failure into verbal feedback;

generative agents [Park et al., 2023] maintain retrieval by recency, importance, and relevance. Externalization work [Zhou et al., 2026] treats skills as procedural templates shared across agents, which KAOS adopts. Unlike systems where an LLM curates its own memory, KAOS’s plasticity layer is deliberately non-linguistic: ranking changes are arithmetic over outcome records, which makes them cheap, auditable, and immune to self-flattering rewrites. Letta’s sleep-time agents [Packer et al., 2023] pursue the complementary LLM-driven approach.

Harness engineering. The term crystallized in early 2026 [Hashimoto, 2026, Osmani, 2026], with the recurring claim that scaffold quality dominates model quality for agentic outcomes. A noted critique of the practice is the absence of verification that a harness change helped [Böckeler, 2026]. Our evaluation discipline (§6) is a direct response. Meta-harness search [Anonymous, 2026d] treats the harness itself as an optimization target; KAOS embeds a variant as a subsystem, and CORAL-style stagnation pivots [Anonymous, 2026a] govern its search loop.

Skill-transfer skepticism. SWE-Skills-Bench [Anonymous, 2026f] reports that 39 of 49 deployed agent skills produced no measurable gain and 3 degraded performance; cross-task episodic memories similarly failed to beat their baselines in any configuration [Anonymous, 2025a]. These negative priors shaped our thresholds: mechanisms in this class must beat the *native* baseline (not an ablated one) with a pre-registered margin, which several proposals in our ledger failed to do (§7).

Evaluation methodology. Our discipline transplants pre-registration and falsification norms [Popper, 1959] into harness engineering: hash-locked analysis plans, mandatory ability-to-fail checks, and no post-hoc retuning. Recent work documents run-to-run variance of 2.2–6.0 pp in pass@1 even at temperature zero [Anonymous, 2026e], motivating the cluster (run-level) bootstrap and binding power budgets KAOS added for multi-run probes; e-process sequential testing [Anonymous, 2025b] is planned for early stopping. Blind judging draws on the surrogate-verifier idea of information-isolated diagnosis [Anonymous, 2026b].

Biological framing. The analogy to neuroplasticity is architectural, not neurobiological: we borrow the separation between fast, local synaptic updates [Hebb, 1949] and slow, offline structural consolidation [Tononi and Cirelli, 2014]. The algorithms themselves are classical—Wilson intervals [Wilson, 1927], exponential decay, co-occurrence counting.

3 The KAOS Substrate

KAOS stores all state for a project in one SQLite database (schema v9; 26 tables, 2 FTS5 indexes; P15): per-agent virtual filesystems with enforced isolation, content-addressed blobs, an append-only event journal recording every tool call with token and cost accounting, checkpoints with byte-exact restore, a cross-agent memory store and skill library (both FTS5/BM25 indexed), and a shared coordination log. Read-only SQL access for agents is enforced at the SQLite authorizer layer. The single-file design is not a scaling liability in the regimes we measured: cross-agent full-text search is 1.5 ms p95 with 1,000 resident agents and 9.2 ms p95 with 10,000 (8 threads, mixed read/write/search/checkpoint workload; P7),² and the dominant write-latency cost proved to be the durability pragma, not data volume: switching from `synchronous=FULL/wal_autocheckpoint=100` to SQLite’s recommended `NORMAL/1000` (the two pragmas changed together; no arm isolates them) moved write p95 from 1,895 ms to 15 ms and throughput from 29 to 1,118 ops/s under contention (P8). We report this partly as a caution: we nearly undertook a per-agent-database redesign before the measurement showed the premise false (P13).

²Claim labels (P1–P16) refer to rows of the provenance ledger in Appendix A.

4 Two-Timescale Plasticity

Every skill application, memory retrieval, and agent completion is a “firing.” The design question is what work may happen at firing time. Our answer, after measurement, is: *almost none*.

Fast path (synaptic timescale). Inline hooks write raw telemetry rows—`skill_uses` (with a binary or graded outcome), `memory_hits` (with query and rank), episode counters—inside the caller’s existing transaction. The hooks that could do more are deliberate no-ops; association building never runs at firing time. A paired micro-benchmark (200 ops, plasticity on vs. off) measures the median hot-path overhead at $-24.6\mu\text{s}$ for outcome recording and $-87.0\mu\text{s}$ for memory search—negative point estimates from a single 200-op paired run, i.e. within measurement noise rather than a tested null result—while the episode-completion hook, which triggers batch work, carries $\approx +1\text{ms}$ median, consistent with the design (P3).

Batch path (consolidation timescale). At agent completion, and every N completions (default 100, tunable via `KAOS_DREAM_THRESHOLD`), a consolidation pass (i) rebuilds the Hebbian co-occurrence graph with set-based SQL—skills and memories that fired within the same episode receive weighted edges with exponential decay on staleness; (ii) refreshes the signals that §5 reads at query time; (iii) generates *proposals*: promote, prune, or merge entries based on usage evidence. Automatic consolidation is proposal-only; structural mutation requires an explicit operator apply. The batch placement is forced by measurement: a dry-run consolidation costs 175 ms at 100 skills but 35.5 s at 10,000 (P6)—three orders of magnitude too slow for any hot path, and comfortably amortized at episode boundaries.

This is the entire sense in which we use the word *neuroplasticity*: telemetry accumulates as a side effect of operation (there is no separate training job to run), and structure changes on a slower, offline clock. We make no stronger biological claim.

5 Usage-Weighted Retrieval

Skill retrieval ranks candidates by

$$\text{score}(s \mid q) = \max(\text{bm25}(s, q), \epsilon) \cdot (c + \alpha \cdot W(s)) \cdot \lambda^{\Delta t(s)}, \quad (1)$$

where `bm25` is lexical relevance over the FTS5 index, $W(s)$ is the Wilson lower bound [Wilson, 1927] of the skill’s success rate over recorded outcomes (graded outcomes contribute their quality value in $[0, 1]$), $\lambda^{\Delta t}$ is exponential recency decay, and α (default 3) weights the usage term; the implementation uses a base weight $c=0.5$ and a relevance floor $\epsilon=0.01$ (`kaos/dream/signals.py`). Weighted scores are computed at *query time* from the accumulated telemetry; the batch pass of §4 maintains the association graph and consolidation proposals, not the ranking itself. The Wilson bound is the load-bearing choice: it discounts small samples, so a skill that succeeded once does not outrank one that succeeded forty times out of fifty.

Benchmarks. Both retrieval benchmarks are small and fully committed, with generation scripts, seeds, and raw outputs in the repository. We consider disclosure of n non-negotiable at this scale and discuss the limitation explicitly in §10. One asymmetry must be stated plainly: the weighted ranker predates the probe discipline of §6 and was evaluated on these committed-but-not-pre-registered benchmarks; mechanisms proposed after v0.9 faced locked gates. A pre-registered, native-baseline probe of Eq. 1 under the multi-run protocol is the first item of queued future work.

Sensitivity. Sweeping α on the realistic benchmark yields 73.3% at $\alpha=0$ (pure BM25), a non-monotone low- α region (86.7% at $\alpha=0.5$, dipping to 80.0% at $\alpha=1$), and a flat 93.3% plateau

Benchmark	Queries / skills / episodes	BM25	Weighted	Δ
Adversarial (twin-pair distractors)	10 / 20 / 80	80.0%	90.0%	+10.0 pp
Realistic (natural phrasing, no twins)	15 / 40 / 120	73.3%	86.7%	+13.3 pp

Table 1: Top-1 retrieval accuracy, BM25 vs. usage-weighted ranking (Eq. 1). Both benchmarks are deliberately small and committed in full (P1, P2); at these sizes the deltas correspond to a net of one flipped query (two gains, one regression) and two flipped queries respectively, and we treat them as evidence of mechanism, not of effect size. Gains require outcome feedback: on single-session workloads with no recorded outcomes the weighted ranker reduces to BM25 (zero gain by construction).

across $\alpha \in \{2, 3, 5, 8, 12\}$ (P5),³ indicating robustness within the $\alpha \geq 2$ operating region (the low- α region is not smooth). A companion experiment replacing binary outcomes with graded quality scores improved mean accuracy +4.0 pp across five seeds while *failing* to confirm our variance-reduction hypothesis (seed-level variance slightly increased); both halves are reported in the committed results (P4).

6 The Falsifiable-Evaluation Discipline

Mechanisms like §5 are easy to justify with a demonstration and hard to justify honestly. The KAOS rule is that no mechanism ships without surviving a probe that could have killed it:

1. **Pre-register.** Kill gates (exact thresholds against the *native* baseline), arm definitions including lesion arms, and workload invariants are written into a lock file whose sha256 is committed before any feature code exists. The harness refuses to run against a lock whose hash is not in its allow-list, making post-hoc goalpost movement mechanically detectable.
2. **Falsify.** A mandatory self-test substitutes the feature arm with its baseline; the kill gate must fire. A harness in which the feature cannot lose is inadmissible, and any later “pass” from it is void.
3. **Run.** Arms execute; outcomes are judged blind (the judge receives an anonymized, shuffled stream and never sees arm identity); gates are computed; the verdict—ACCEPT, REJECT, or VOID—is binding, with no retune-and-rerun.
4. **Journal.** Every run is appended to an experiments table with the git revision, lock hash, per-arm aggregates, and verdict.

For multi-run probes, gates use a cluster (run-level) bootstrap rather than a within-run bootstrap, and a lock-declared runs-per-condition is binding: executing fewer runs voids the verdict rather than silently widening an interval. This responds directly to measured run-to-run variance in LLM evaluation [Anonymous, 2026e].

³The sweep’s plateau accuracy (93.3%) exceeds Table 1’s weighted accuracy (86.7%) at the same nominal configuration because the sweep script re-implements the weighted scorer locally rather than calling the production `SkillStore.search` path; the two code paths produce different episode trajectories. Both artifacts are committed; the sweep therefore characterizes the ranking *rule*’s α -sensitivity, not the shipped implementation, and a rerun through the production path is queued.

Under this discipline, ten mechanism candidates were evaluated between v0.7 and v0.10. None shipped without an ACCEPT; the ledger (P16) records two probe REJECTs, one structural DO-NOT-BUILD, one VOID, three literature-prior rejections, and three parks. We describe the informative ones next.

7 Negative Results

7.1 The lexical vise: semantic consolidation is infeasible here

The most attractive mechanism we evaluated was memory consolidation in the strong sense: periodically synthesize clusters of related memories into abstracted insights, store the insights, and retrieve them for compositional queries—the standard “sleep consolidates memories into knowledge” story [Anonymous, 2026g].

The pre-registered probe rejected it, and the failure mode is more interesting than the failure. In the LLM arm, synthesized insights *were retrieved*—present in the top- k for roughly half the queries they should have helped—yet accuracy on the hard query classes was 0.000 (P9): the synthesizer produced correct bridging prose while paraphrasing away the exact technical tokens (identifiers, error strings, config keys) that downstream exact-match evidence required. A follow-up extractive variant, pre-registered before its code existed, inverted the failure: by construction it preserved decisive tokens verbatim (44/44 in its token-fidelity check) but, containing only vocabulary from its source memories, was never retrieved for the semantically distant queries it was meant to serve—scoring 0.000, identical to raw BM25 (P10).

The general form is a two-horned constraint we call the *lexical vise*: under lexical-only retrieval, a stored abstraction must be simultaneously (a) token-faithful to its sources, or downstream verification fails, and (b) lexically bridged to future queries, or it is never retrieved. Generative synthesis achieves (b) at the cost of (a); extraction achieves (a) at the cost of (b). The two oppositely-designed variants we tested each impaled on one horn; untested hybrids (e.g. extractive spines with appended paraphrase expansions) would require a fresh pre-registered lock under the repository’s rejected-class rule, and we conjecture—but have not proven—that any lexical-only variant faces the same trade-off. On that basis we closed the mechanism class for this architecture, which we regard as the most valuable single output of the discipline: it converts an open-ended research direction into a bounded infeasibility argument, supported by two pre-registered failures, for a specific and common constraint regime.

7.2 A VOID that stayed void

A pre-execution action-validation layer (motivated by Anonymous, 2026c) was probed with gates locked before code, including a workload invariant: at least 200 *organic* action-class failure incidents, with synthetic substitution explicitly forbidden. The development database contained two. The verdict was VOID—not rescued by generating synthetic incidents, not downgraded to a demo (P11). We highlight this because the temptation the lock removed (“just simulate the workload”) is precisely how plausible-but-unverified mechanisms normally enter frameworks. The falsification self-test for this probe passed (the harness could kill the feature); the evidence to judge the feature simply did not exist yet.

7.3 The instrument was buggy, and the process caught it

In June 2026 we pointed the evaluation machinery at KAOS itself: a 48-agent scoping workflow (six subsystem code reviewers, five literature scouts, and a 36-judge panel over three lenses) with a requirement that findings cite the file and line they read. The panel found, among storage-layer defects, three bugs in the verdict instrument: the blind judge’s agreement statistic was computed as an identity comparison (always 1.0—a dead audit gate); a probe registering zero kill gates would vacuously ACCEPT; and the verdict re-verification path crashed on the very results files the reference probe produced. All were fixed test-first, and the retroactive check matters: no prior verdict flips under the corrected instrument, because the REJECT margins were far outside any plausible correction and the VOID was an n -floor decision (P12). An independent external model review later confirmed two documentation overstatements (both fixed same-day) while its strongest claims did not survive comparison with source. We include this subsection because a discipline that cannot describe its own failures is exactly the kind of unfalsifiable system it exists to prevent.

8 Field Observation: ARC-AGI-3

Between June 8 and July 18, 2026, a KAOS-operated entry in the ARC-AGI-3 public leaderboard moved from score 0.43 (rank #47 of $\sim 1,100$) through 0.93 (#219 of 1,680, after the field grew $\sim 55\%$) and 1.02 (#177 of 1,693) to rank #39 of $\sim 1,700$ —the top 2.3% (P14). The iteration loop—strategy proposal, evaluation, skill retention, submission—was the harness’s meta-harness subsystem; the operator adjusted configuration between runs. We deliberately present this as an *observation*, not a result: there is no counterfactual arm, the public leaderboard is computed on partial test data, and operator interventions are not controlled. Its role in this paper is motivational: it is what a parameter-free, usage-statistics harness looks like when pointed at a live competition, and it is externally auditable in a way private demos are not.

9 Discussion

What usage statistics buy. Within their scope, the measured gains are real but modest, and they are conditional: every mechanism in §5 requires outcome feedback to exist. The honest summary is that usage-statistics plasticity converts *repeated, feedback-rich* workloads into ranking improvements at near-zero marginal runtime cost, and does nothing for single-shot workloads. We consider the near-zero fast path the more important property: it makes the mechanism safe to leave on.

What the discipline buys. Four of the five most consequential findings in this paper are rejections or corrections. A field survey of our own ledger suggests the base rate of plausible harness mechanisms that survive a native-baseline, pre-registered probe is low—consistent with external evidence on deployed skills [Anonymous, 2026f]. If that base rate is real, then evaluation infrastructure with teeth is not overhead on harness engineering; it is most of the substance, and the observed tendency of documentation to overstate what code honestly implements (§7.3) is an argument for making the instrument, not the prose, the source of truth.

Auditability as a design axis. Because all plasticity state is rows in one inspectable database—with an append-only journal and authorizer-enforced read-only access—every ranking decision in §5 can be reconstructed after the fact with SQL. We found this property repeatedly load-bearing: for the self-audit, for responding to external review, and for writing this paper, whose every number was re-derived from the committed artifacts (Appendix A).

10 Limitations

The retrieval benchmarks are small ($n=10$ and $n=15$ queries; the deltas are a net one and two flipped queries, including one regression on the adversarial bench) and synthetic; they establish mechanism, not effect size, and a larger- n replication under the cluster-bootstrap protocol is queued but not done. The overhead and scale measurements are single-machine (Windows/NTFS) and workload-specific. The lexical-vise argument (§7.1) is an engineering impossibility argument for a specific constraint regime—lexical retrieval, exact-match downstream evidence, no parameter updates—supported by two pre-registered failures, not a formal theorem; systems that relax any of the three constraints escape it. The ARC-AGI-3 trajectory is observational with operator involvement and partial-data scoring, and supports no causal claim. The quality-score experiment’s variance hypothesis failed. Parts of the plasticity loop reached the primary MCP surface only in v0.10; earlier deployments under-collected telemetry. Finally, the multi-agent panels used to review this paper share a model family with its drafting agent; human review was the final gate, but panel independence is bounded by that fact.

11 Meta: How This Paper Was Produced

Consistent with the system it describes, this paper was drafted by an agent, against the provenance ledger of Appendix A, and then passed through three KAOS-orchestrated panel levels before human sign-off: (i) a grounding panel that re-verified every quantitative claim against the committed artifacts; (ii) a rigor panel reviewing statistical claims, reproducibility of the method descriptions, and biological-analogy overreach; (iii) an adversarial panel simulating hostile peer review. Panel verdicts and the resulting revisions are recorded in the repository alongside the source (`paper/PANEL_LEDGER.md`). The final responsibility for all claims rests with the human author.

12 Conclusion

A parameter-free agent harness can obtain measurable, auditable adaptivity from usage statistics alone, provided structural work is exiled from the hot path—and keeping such a system honest requires evaluation machinery with the power to reject its authors’ ideas, which it will exercise regularly. Future work: a larger- n retrieval replication under the multi-run protocol; e-process early stopping for probes; a signed, capability-declared package layer for federating skills across organizations on the same single-file substrate; and re-running the action-validation probe when sufficient organic evidence accumulates.

Acknowledgments and AI-use disclosure

This paper was drafted and revised with agentic assistance from Claude (Anthropic), orchestrated through KAOS itself, including the multi-level panel review described in §11; all quantitative content derives from the committed artifacts of Appendix A, and responsibility for all claims rests with the human author.

Reproducibility

All code, benchmarks, locks, verdicts, and the provenance ledger are at <https://github.com/canivel/kaos> (MIT). Each benchmark ships a `run.py` regenerating its committed results file.

References

- Anonymous. CTIM-Rover: Cross-task instance memory for repository-level agents. *arXiv preprint arXiv:2505.23422*, 2025a.
- Anonymous. E-evaluators: Anytime-valid sequential testing for model evaluation. *arXiv preprint arXiv:2512.03109*, 2025b.
- Anonymous. CORAL: Towards autonomous multi-agent evolution for open-ended discovery. *arXiv preprint arXiv:2604.01658*, 2026a.
- Anonymous. EvoSkills: Skill evolution with surrogate verifiers. *arXiv preprint arXiv:2604.01687*, 2026b.
- Anonymous. Adapting the interface, not the model: Runtime harness adaptation for deterministic LLM agents. *arXiv preprint arXiv:2605.22166*, 2026c.
- Anonymous. Meta-harness: Evolutionary search over agent scaffolds from execution traces. *arXiv preprint arXiv:2603.28052*, 2026d.
- Anonymous. Run-to-run variance in LLM evaluation: Why single-run pass@1 misleads. *arXiv preprint arXiv:2602.07150*, 2026e.
- Anonymous. SWE-Skills-Bench: Do deployed agent skills actually help? *arXiv preprint arXiv:2603.15401*, 2026f.
- Anonymous. Contextual agentic memory is a memo, not true memory. *arXiv preprint arXiv:2604.27707*, 2026g.
- Birgitta Böckeler. Harness engineering — first thoughts. <https://martinfowler.com/articles/exploring-gen-ai/harness-engineering-memo.html>, 2026. Memo, February 17, 2026.
- Mitchell Hashimoto. My AI adoption journey. <https://mitchellh.com/writing/my-ai-adoption-journey>, 2026. Blog post, February 5, 2026; origin of the term “harness engineering”.
- Donald O. Hebb. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, 1949.
- Addy Osmani. Agent harness engineering. <https://addyosmani.com/blog/agent-harness-engineering/>, 2026. Blog post, April 19, 2026.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.
- Karl R. Popper. *The Logic of Scientific Discovery*. Hutchinson, 1959.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.

- Giulio Tononi and Chiara Cirelli. Sleep and the price of plasticity: From synaptic and cellular homeostasis to memory consolidation and integration. *Neuron*, 81(1):12–34, 2014.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- Wei Zhou et al. Externalization in LLM agents: Procedural skill libraries across agent populations. *arXiv preprint arXiv:2604.08224*, 2026.

A Provenance Ledger

Every quantitative claim in this paper carries a label P1–P16 mapping to a row of `paper/PROVENANCE.md` in the repository, which lists the committed artifact (results file, lock file with sha256, or commit hash) from which the number derives. Pre-registration hashes cited: synthesis-consolidation lock v2 09310794...; action-realization lock 3ca89983... Repository state for suite-level counts: commit 8783871 (701 tests passing; 58 MCP tools; schema v9).